

Lecture – 3.3

JAVA variables and Modifiers

Afsara Tasneem Misha

Lecturer

Department of CSE

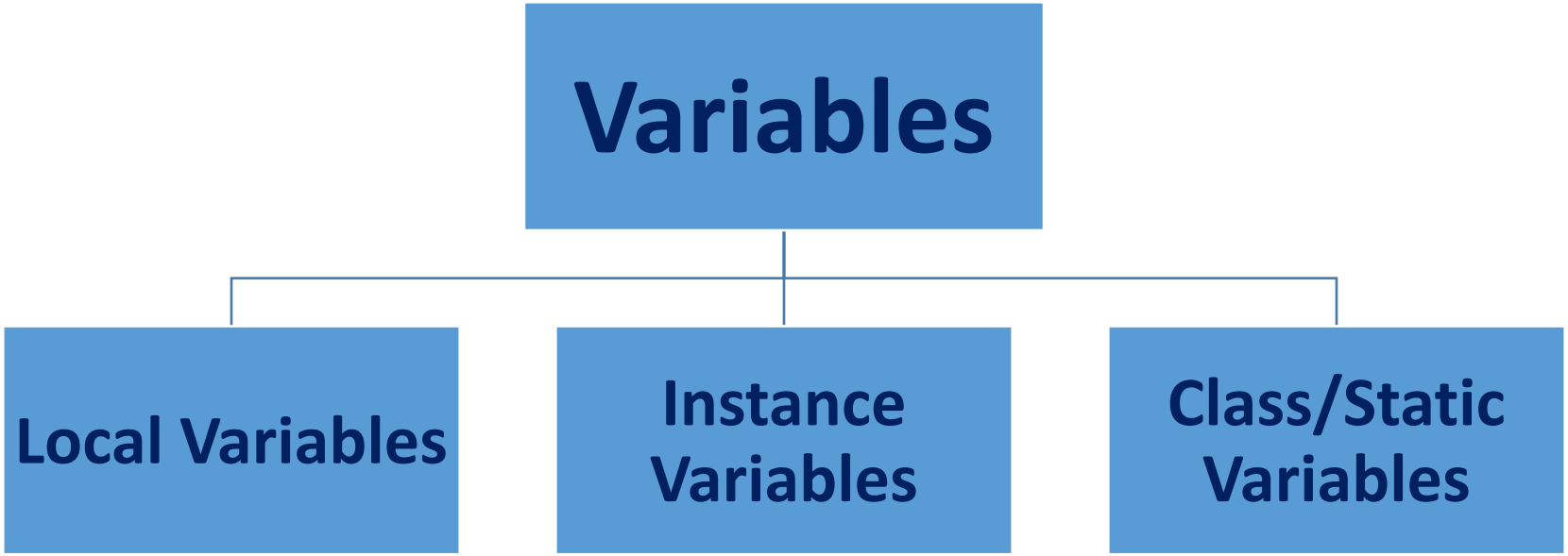
Daffodil International University

Contents

- Java variables
- Java Modifiers

Variables in java

Variables in java



1. Local variables

- Local variables are declared **inside methods, constructors, or blocks**.
- Local variables are **visible only within** the declared method, constructor or block.
- **Access modifiers cannot** be used for local variables.

2. Instance variables

- Instance variables are declared **in a class**, but **outside** a method, constructor or any block.
- Instance variables are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed.
- **Access modifiers can be given** for instance variables.
- The instance variables are **visible** for **all methods**, **constructors** and **block** in the class.

3. Class/static variables

- Class variables also known as **static variables** are declared with the ***static*** **keyword** in a class, but **outside** a **method**, **constructor** or a **block**.
- Static variables are rarely used other than being declared as **constants**.
- Visibility is **similar** to **instance variables**.
- However, most static variables are declared **public** since they must be available for users of the class.

Key points

- **Local Variable**

  A variable that is declared inside the method

- **Instance Variable**

  A variable that is declared inside the class but outside the method.

  It is not declared as static.

- **Static variable**

  Similar to instance variable but it is declared as static.

Example to understand the types of variables

Example - 1:

```
class A
{
    int data=50;          //  instance variable
    static int m=100;     //  static variable

    void method()
    {
        int n=90;         //  local variable
    }

}
```

Example - 2:

```
public class MyClass
{
        int a = 50;
        static int b = 100;

        public void method1()
        {
                int n = 90;
                System.out.println(n);
                System.out.println(a);
                System.out.println(b);
        }

        public void method2()
        {
                System.out.println(n);
                System.out.println(a);
                System.out.println(b);
        }

        public static void main(String[] args)
        {
                MyClass ob = new MyClass ();
                 ob.method1 ();
                 ob. method2 ();
        }

}
```

Example - 3:

```
public class NewClass {

    int max = 100;   //instance variable
    static int var = 50;    // static variable

    public static void main(String[] args)
    {
        int a = 10, b = 20;      // local variable
        System.out.println(a+b);
        NewClass obj = new NewClass();
        System.out.println(obj.max);
        System.out.println(var);
        sum();
    }
    public static void sum()
    {
        NewClass obj = new NewClass();
        System.out.println(obj.max);
        System.out.println(var);
        //System.out.println(a+b);
    }
}
```

Modifiers in java

Modifiers

Modifiers are **keywords** that you add to those definitions to change their meanings.

There are **two** types of modifiers in java:

1. Access modifiers
2. Non-access modifiers

1. Access Modifiers

1. Access Modifiers in java

- The access modifiers in java specifies **accessibility** (scope) of a **data member**, **method**, **constructor** or **class**.

- There are **4 types** of java access modifiers:

1. **private**
2. **default**
3. **protected**
4. **public**

1) private access modifier

- The private access modifier is accessible only **within the class.**

Simple example of private access modifier

In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is compile time error.

```
class A{
private int data=40;
private void msg(){System.out.println("Hello java");}
}

public class Simple{
 public static void main(String args[]){
   A obj=new A();
   System.out.println(obj.data);
   obj.msg();//Compile Time Er
   }
}
```

private access modifier

```
class A{
private int data=40;
private void msg(){System.out.println("Hello java");}
}

public class Simple{
 public static void main(String args[]){
   A obj=new A();
   System.out.println(obj.data);//Compile Time Error
   obj.msg();//Compile Time Error
   }
}
```

2) default access modifier

- If you don't use any modifier, it is treated as **default** by default.

- The default modifier is accessible **only within package**.

Example of default access modifier

In this example, we have created two packages pack and mypack. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside the package.

In the above example, the scope of class A and its method msg() is default so it cannot be accessed from outside the package.

```java
//save by A.java
package pack;
class A{
  void msg(){System.out.println("Hello");}
}
```

```java
//save by B.java
package mypack;
import pack.*;
class B{
  public static void main(String args[]){
   A obj = new A();
   obj.msg();
  }
}
```

default access modifier

```java
//save by A.java
package pack;
class A{
  void msg(){System.out.println("Hello");}
}
```

```java
//save by B.java
package mypack;
import pack.*;
class B{
  public static void main(String args[]){
   A obj = new A();//Compile Time Error
   obj.msg();//Compile Time Error
  }
}
```

3) protected access modifier

- The **protected access modifier** is **accessible within package** and **outside** the package but through **inheritance only**.

- The protected access modifier can be applied on the **data member**, **method and constructor**. It can't be applied on the **class**.

- **Example of protected access modifier**

- In this example, we have created the two packages **pack** and **mypack**. The A class of pack package is public, so can be accessed from outside the package. But **msg** method of this package is declared as protected, so it can be accessed from outside the class only through inheritance.

```
//save by A.java
package pack;
public class A{
protected void msg(){System.out.println("Hello");}
}
```

```
//save by B.java
package mypack;
import pack.*;

class B extends A{
  public static void main(String args[]){
   B obj = new B();
   obj.msg();
  }
}
```

protected access modifier

```java
//save by A.java
package pack;
public class A{
protected void msg(){System.out.println("Hello");}
}
```

```java
//save by B.java
package mypack;
import pack.*;

class B extends A{
  public static void main(String args[]){
   B obj = new B();
   obj.msg();
  }
}
```

```
Output:Hello
```

4) public access modifier

- The **public access modifier** is accessible **everywhere**.
- It has the **widest** scope among all other modifiers.
- **Example of public access modifier**

```
//save by A.java

package pack;
public class A{
public void msg(){System.out.println("Hello");}
}
```

```
//save by B.java

package mypack;
import pack.*;

class B{
  public static void main(String args[]){
   A obj = new A();
   obj.msg();
  }
}
```

public access modifier

```
//save by A.java

package pack;
public class A{
public void msg(){System.out.println("Hello");}
}
```

```
//save by B.java

package mypack;
import pack.*;

class B{
  public static void main(String args[]){
   A obj = new A();
   obj.msg();
  }
}
```

```
Output:Hello
```

Understanding all java access modifiers

Let's understand the access modifiers by a simple table.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

2. Non-Access Modifiers

2. Non-Access Modifiers

- ***static* modifier**
  - for creating class methods and variables

- ***final* modifier**
  - for finalizing the implementations of classes, methods, and variables.

- ***abstract* modifiers**
  - for creating abstract classes and methods.

- ***synchronized* and *volatile* modifiers**
  - which are used for threads.

Thank you!

